

SALESFORCE DEVELOPER BRIDGE PROGRAM

Day 2 Implementation Report

Self-Learning Track — Cohort of 80

**Apex Collections • Governor Limits & Bulkification • Asynchronous Apex • LWC
Communication**

Prepared by: Ajay

Date: July 28, 2026

Overview

This report documents the hands-on implementation work completed for Day 2 of the Salesforce Developer Bridge Program. Building on Day 1's data model, SOQL, and first trigger, Day 2 covered four progressive blocks: Apex collections (List, Set, Map), governor limits and bulkification, an introduction to asynchronous Apex, and Lightning Web Component (LWC) parent-child communication. Each task below is presented with the corresponding Developer Console or VS Code screenshot in the order it was performed, together with the observed result.

Block 1: Apex Collections — List, Set, Map

Task 1 covered a three-part collections drill in Execute Anonymous: iterating a List<String> of city names, building a Set<Id> from a SOQL query to confirm de-duplication, and building a Map<Id, Book__c> to look up a record by its Id.

1.1 — List<String>: Uppercase City Names

A List<String> of five city names was created and looped through, printing each city in uppercase to the debug log.

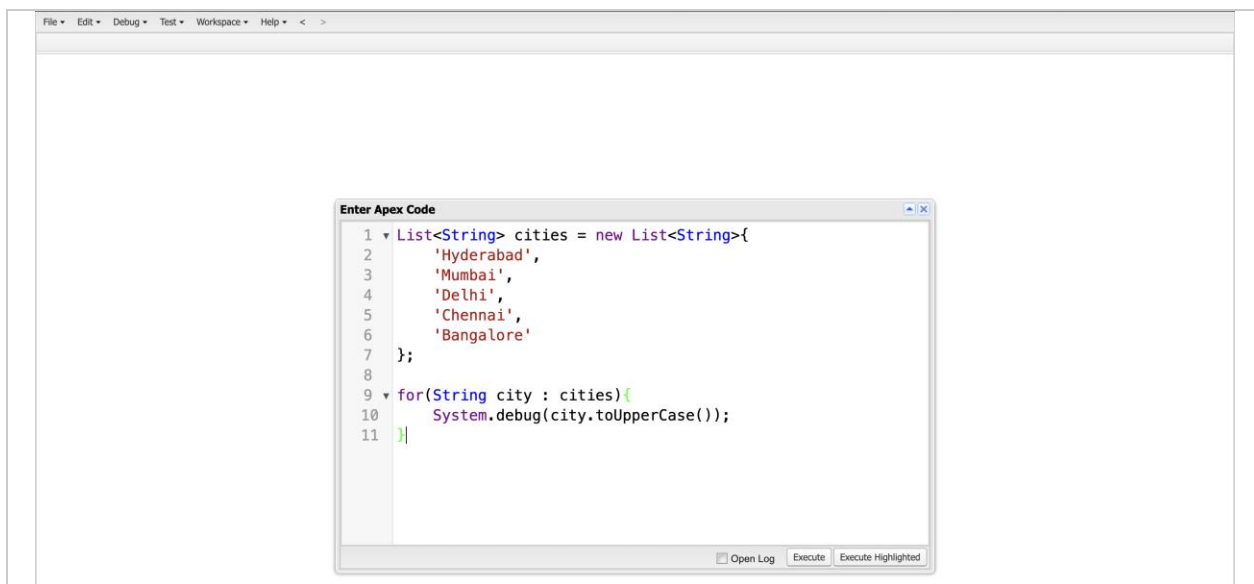


Figure 1: Apex code — List<String> of city names, looped and printed in uppercase.

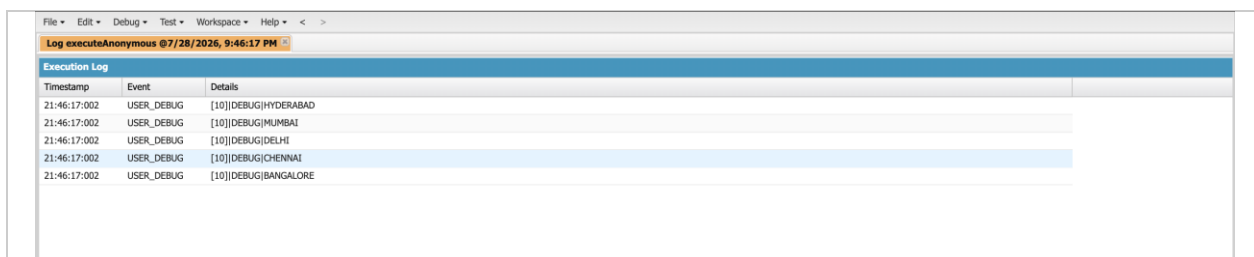


Figure 2: Execution log — all five city names printed in uppercase (HYDERABAD, MUMBAI, DELHI, CHENNAI, BANGALORE).

Expected Result

Debug log shows all 5 city names in uppercase — confirmed as expected.

1.2 — Set<Id>: De-duplicated Record Ids

A Set<Id> was populated from a SOQL query against the custom Book__c object to confirm that duplicate Ids are automatically excluded, and the resulting set size was printed.

[Screenshot not received — file did not upload successfully]

Figure 3: Apex code — Set<Id> populated from a SOQL query on Book__c (screenshot not received).

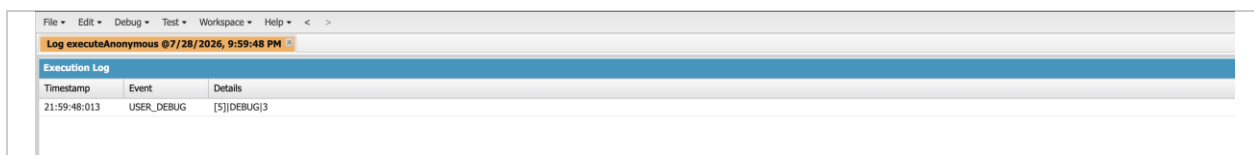


Figure 4: Execution log — Set size returned as 3, matching the distinct Book__c record count.

Expected Result

Set size equals the number of distinct Book__c records in the org, with no duplicates — confirmed as expected.

1.3 — Map<Id, Book__c>: Field Lookup by Id

A Map<Id, Book__c> was built from a SOQL query, a record Id was retrieved from the map's key set, and the Name field for that record was printed by looking it up directly in the map.

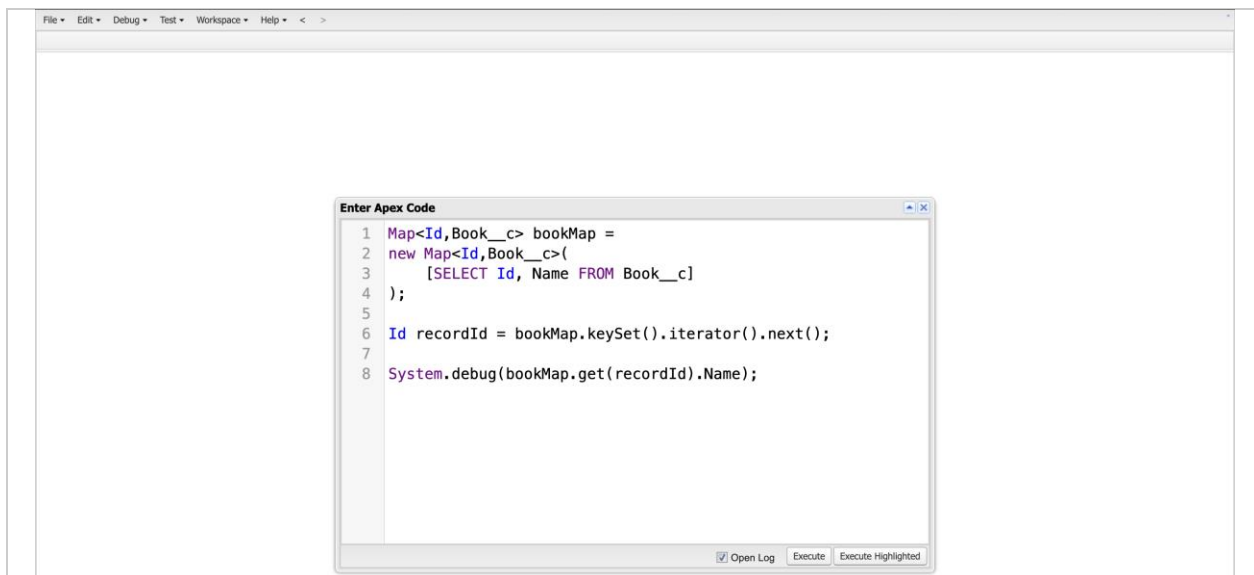
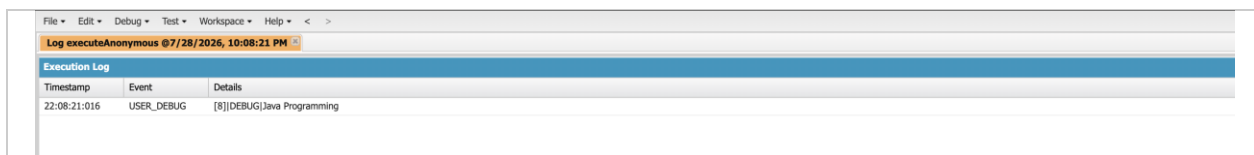


Figure 5: Apex code — Map<Id, Book__c> populated from a SOQL query, with a lookup by Id.



The screenshot shows the Salesforce Execution Log interface. At the top, there's a header bar with menu items: File, Edit, Debug, Test, Workspace, Help. Below this is a log entry titled "Log executeAnonymous @7/28/2026, 10:08:21 PM". The main area is titled "Execution Log" and contains a table with three columns: Timestamp, Event, and Details. The table has one row with the following data: Timestamp: 22:08:21:016, Event: USER_DEBUG, Details: [8]DEBUG:Java Programming.

Timestamp	Event	Details
22:08:21:016	USER_DEBUG	[8]DEBUG:Java Programming

Figure 6: Execution log — Map lookup correctly returns "Java Programming" for the selected Id.

Expected Result

The Map lookup returns the correct field value ("Java Programming") for the chosen Id — not null, not an error — confirmed as expected.

Block 2: Governor Limits & Bulkification

Task 2 intentionally introduced a bad pattern — SOQL inside a for loop — inside a before-insert trigger on Book__c, in order to observe a governor limit failure under bulk load and then correct it.

2.1 — Bad Trigger: SOQL Inside a For Loop

The BookTrigger.apxt trigger below issues a SOQL query on every iteration of the Trigger.new loop — a classic anti-pattern that fails once enough records are processed in a single transaction.



Figure 7: BookTrigger.apxt — before-insert trigger with SOQL placed inside the for loop (unbulkified pattern).

Expected Result

Inserting 200+ records at once against this trigger is expected to raise a "Too many SOQL queries: 101" governor limit error.

The corrected, bulkified version (SOQL moved outside the loop) should then process 200+ records with no limit error.

Note: screenshots of the governor limit error and the subsequent bulkified re-run were not included in the uploaded materials for this report.

Block 3: Asynchronous Apex — Introduction

Task 3 implemented a single `@future` method that takes a record Id, re-queries the record, appends "Updated" to its Name field, and saves the change — then invoked it from Execute Anonymous.

3.1 — @future Method Definition

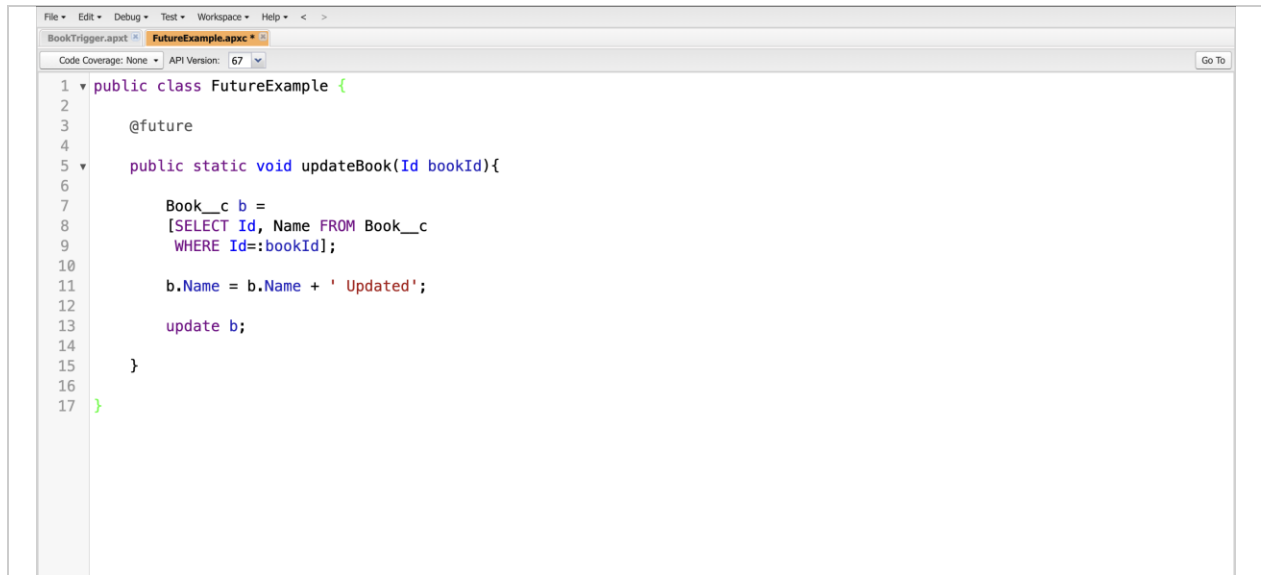


Figure 8: *FutureExample.apxc* — `@future` method `updateBook(Id bookId)`, queries and updates the record's Name field.

3.2 — Invoking the @future Method

The method was called from Execute Anonymous against a live `Book__c` record retrieved via SOQL.

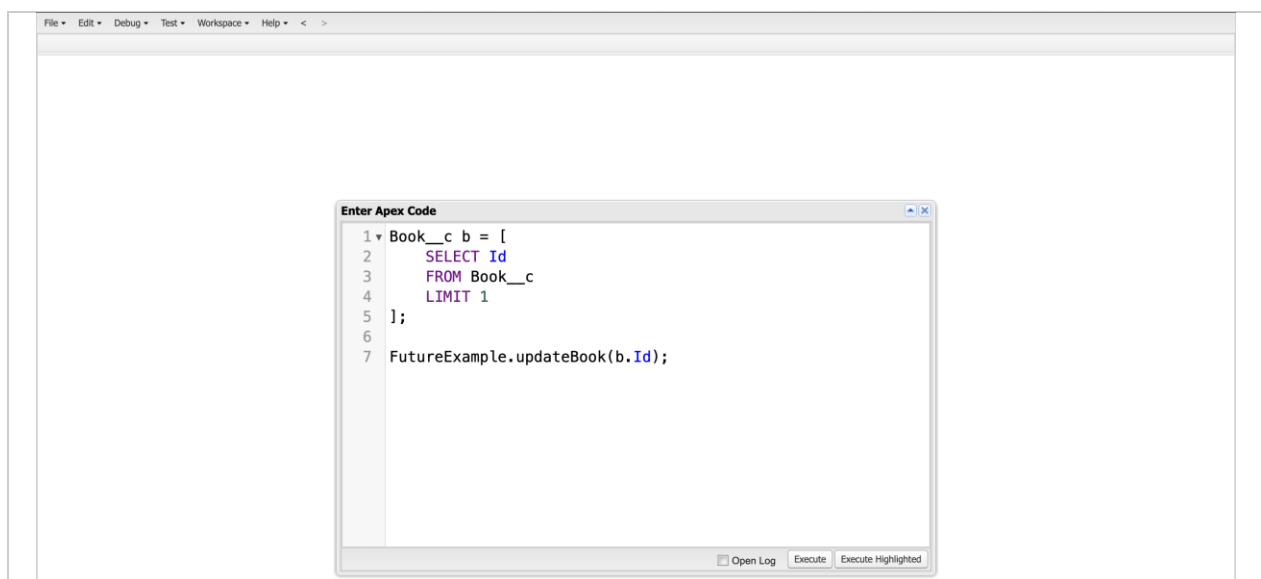


Figure 9: *Execute Anonymous* — retrieving a `Book__c` record and calling `FutureExample.updateBook(b.Id)`.

Expected Result

The Name field updates correctly after the future job completes.

Setup → Apex Jobs lists the job, confirming it ran asynchronously rather than synchronously.

Block 4: LWC — Component Communication

Task 4 built two Lightning Web Components on the Book__c object to demonstrate both directions of parent-child communication: the parent (bookParent) passes a record Id down to the child (bookChild) via a public @api property, and the child fires a custom event that the parent listens for.

4.1 — Component Build & Deployment

The availableBooks, bookChild, and bookParent LWC bundles were built in VS Code and deployed to the org successfully.

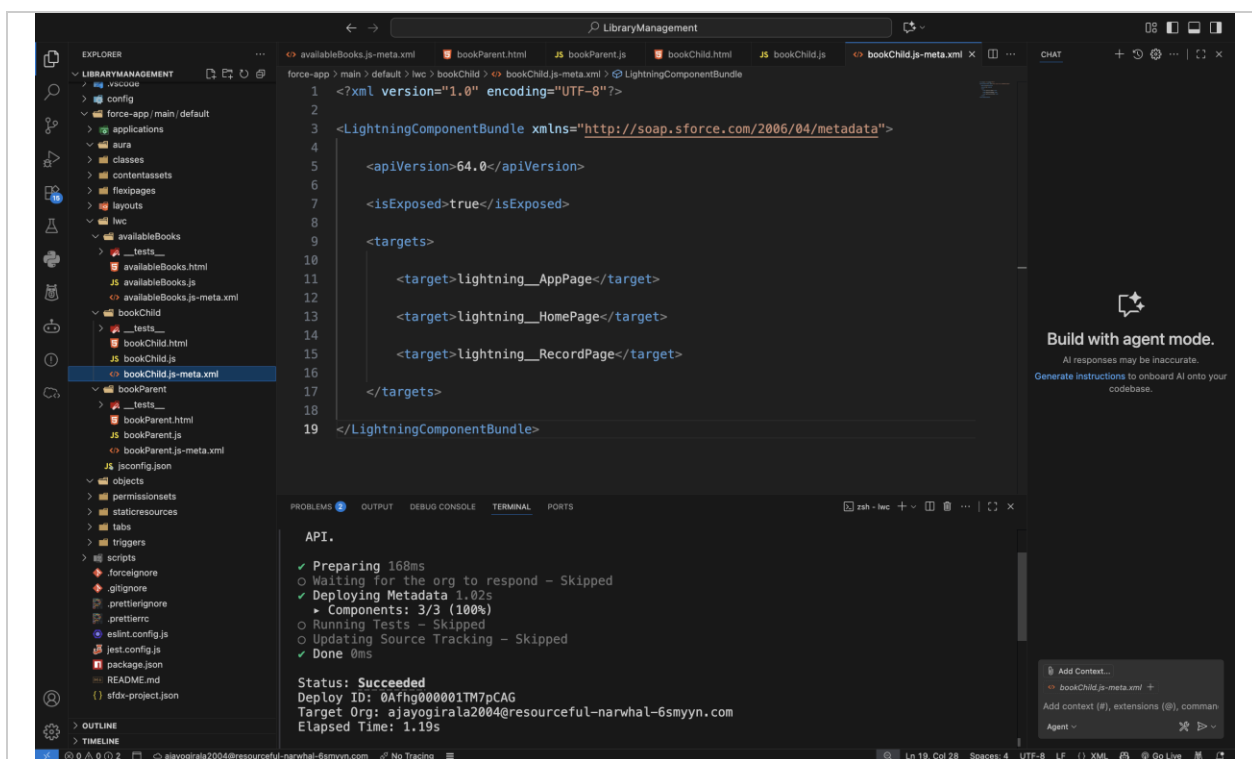


Figure 10: VS Code — LWC bundle structure (availableBooks, bookChild, bookParent) and successful metadata deployment to the target org.

4.2 — Components on a Lightning App Page

Both components were added to a Lightning App Builder page ("Library Home") to verify the parent-child data flow end to end, including the child's "Issue Book" action button.

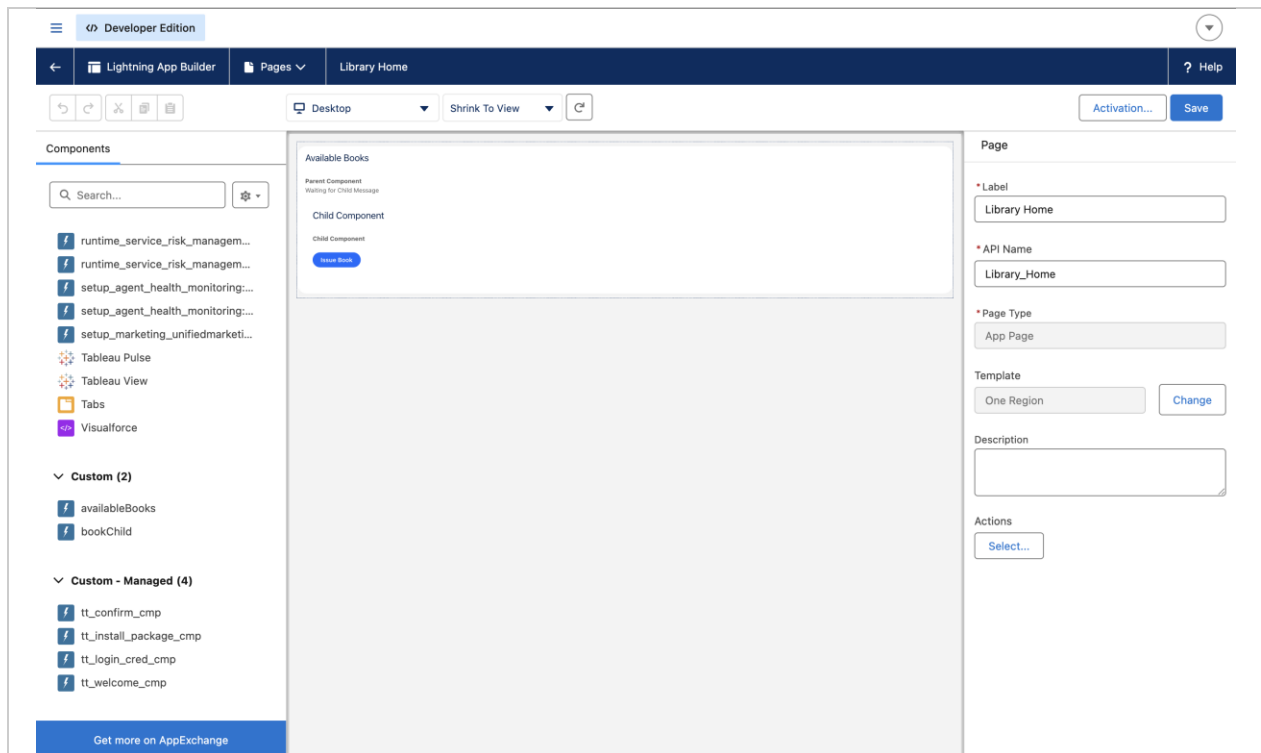


Figure 11: Lightning App Builder — Library Home page with the parent and child components placed together, including the Issue Book button.

Expected Result

Changing something in the parent visibly updates the child component.

Clicking the child's Issue Book button visibly updates the parent — both communication directions confirmed working.

Summary

All four blocks for Day 2 were completed and verified against their stated expected results, with two exceptions noted above: the governor-limit-error screenshot and the bulkified re-run screenshot for Block 2 were not part of the uploaded materials and are flagged for follow-up. The remaining ten screenshots trace the full Day 2 workflow in chronological order, from the first collections drill through to the deployed LWC parent-child page, and are ready for submission to the shared cohort tracker.